



D7.4

Inspection Testbed

Implementation

**Collective awareness, task allocation, and
collision avoidance in Aerostack2**

Guillermo GP-Lenza (UPM)

CORESENSE
Horizon Europe Project #101070254
HORIZON-CL4-2021-DIGITAL-EMERGING-01-11



CORESENSE



CORESENSE

The **CORESENSE Project** is a four year research project focused in the development of a *technology of machine understanding* for the construction of dependable autonomous systems. It is a project funded by the European Union.

Programme: Horizon Europe
Project number: 101070254
Project name: CoreSense: A Hybrid Cognitive Architecture for Deep Understanding
Project acronym: CORESENSE
Topic: HORIZON-CL4-2021-DIGITAL-EMERGING-01-11
Type of action: HORIZON Research and Innovation Actions
Granting authority: European Commission-EU
Project start date: 1 October 2022
Project end date: 30 September 2026
Project duration: 48 months

Most of CORESENSE results are open science but IPR regulations may still hold. Requests for permission to reproduce this document or to prepare derivative works of this document should be addressed to the CORESENSE Consortium IPR management.

Copyright © 2022-2026 by The CORESENSE Consortium.

UNIVERSIDAD POLITECNICA DE MADRID (UPM)

TECHNISCHE UNIVERSITEIT DELFT (TUD)

FRAUNHOFER GESELLSCHAFT ZUR FORDERUNG DER ANGEWANDTEN FORSCHUNG (FHG)

UNIVERSIDAD REY JUAN CARLOS (URJC)

PAL ROBOTICS SL (PAL)

IRISH MANUFACTURING RESEARCH COMPANY LIMITED BY GUARANTEE (IMR)

CZECH TECHNICAL UNIVERSITY IN PRAGE (CVUT)



For information about other CORESENSE products, reports or articles, please visit our Web site:

<http://www.coresense.eu/>

NO WARRANTY — Please read the fine print:

THIS CORESENSE MATERIAL IS FURNISHED ON AN “AS-IS” BASIS. THE CORESENSE CONSORTIUM NOR ANY OF THE INDIVIDUAL CORESENSE PARTNERS OR RESEARCHERS MAKE NO WARRANTIES OF ANY KIND, EITHER EXPRESSED OR IMPLIED, AS TO ANY MATTER INCLUDING, BUT NOT LIMITED TO, ACCURACY, COMPLETENESS, WARRANTY OF FITNESS FOR PURPOSE OR MERCHANTABILITY, EXCLUSIVITY, OR RESULTS OBTAINED FROM USE OF THE MATERIAL. CORESENSE DOES NOT MAKE ANY WARRANTY OF ANY KIND WITH RESPECT TO FREEDOM FROM PATENT, TRADEMARK, OR COPYRIGHT INFRINGEMENT.



**Funded by
the European Union**

The CORESENSE project is funded by the EC Horizon Europe programme through grant HE #101070254 inside the HORIZON-CL4-2021-DIGITAL-EMERGING-01-11 topic. Views and opinions expressed in this document are however those of the author(s) only and do not necessarily reflect those of the European Union or the Horizon Europe Programme. Neither the European Union nor the granting authority can be held responsible for them.

Executive Summary

Title: D7.4 Inspection Testbed Implementation

Subtitle: Collective awareness, task allocation, and collision avoidance in Aerostack2

Author: Guillermo GP-Lenza (UPM)

Reference: CS-D7.4 v 0.1

Date: 2026/04/28

Dissemination level: Public

URL: <http://www.coresense.eu/doc/CS-D7.4.pdf>

This document reports on the implementation of the CORESENSE Drone Inspection Testbed (TB2), describing the software contributions made to the Aerostack2 robotics framework that enable a fleet of autonomous drones to perceive their shared environment, allocate inspection targets, and navigate safely in the presence of peers.

The implementation is grounded in the collective awareness architecture defined in D7.3 [Fernandez-Cortizas et al., 2023]. According to that document, *awareness for a multi-robot system S is the synchronous understanding of the situation by S* . Achieving this requires agents to maintain self-models and collective models, communicate modellets between each other, and integrate information into shared knowledge structures. The CORESENSE definition of cognitive modules—*subsystems that realise the modellet-engine concept, using representations (models) to derive relevant information by suitable algorithms (engines)*—maps naturally onto Aerostack2 *behaviors*, the plugin-based action-server components that encapsulate robot skills.

The document is structured around the infrastructure-first narrative proposed in D7.3: first the communication backbone, then the knowledge layer, and finally the behaviors that use both.

- **Collective Awareness middleware (as2_ca)** — implements the Communication Interface subsystem of the D7.3 Collective Awareness Module: a gateway node that handles peer discovery and modellet routing, and a client component that gives behaviors a typed API for sending and receiving modellets.
- **Knowledge Base Interface (as2_core and as2_python_api)** — connects Aerostack2 behaviors and mission scripts to the CORESENSE `knowledge_core`, enabling fact assertion, querying, and reactive event handling.
- **Auction Behavior (as2_behaviors)** — implements the *auctions* collective model: task allocation through bid exchange over the CA gateway, cost computation from self-model data, and result grounding in the knowledge base.
- **Collision Avoidance Behavior (as2_behaviors)** — implements the *consensus* collective model: distributed path-lock arbitration through lock-request exchange over the CA gateway, with path prepended from self-model data.

Together, these components satisfy the functional requirements for collaborative planning, failure resilience, trajectory collective-awareness, and safety distance maintenance stated in D7.2 [Fernandez-Cortizas et al., 2023c], and enable the inspection scenarios defined in D7.1 [Fernandez-Cortizas et al., 2023a].

We acknowledge the financial support through the contract #101070254, *CoreSense: A Hybrid Cognitive Architecture for Deep Understanding* granted by the European Commission inside the Horizon Europe programme.

Document Versions

Version	0.1 2026/04/28
<i>Changes</i>	Initial draft
<i>Authors</i>	G. GP-Lenza (UPM)

Contents

Executive summary	3
Document Versions	5
1 Introduction	9
1.1 Context and motivation	9
1.2 Aerostack2 and the cognitive module correspondence	9
1.3 Document structure	10
2 Behaviors as Cognitive Modules	11
2.1 The modelet-engine concept in CORESENSE	11
2.2 Aerostack2 behaviors as cognitive module realizations	11
3 Collective Awareness Middleware	13
3.1 The Communication Interface subsystem	13
3.2 Message types: the modelet vocabulary	13
3.2.1 Transport envelope	13
3.2.2 Auction modelets	14
3.2.3 Path-lock modelets	14
3.3 CA Gateway node	14
3.4 CA Gateway Client	15
4 Knowledge Base Interface	17
4.1 C++ KBInterface	17
4.2 Python KB Monitor	17
4.2.1 KBMonitorNode	17
4.2.2 KBMissionMonitor	18
5 Auction Behavior	19
5.1 Overview: auctions as a collective model	19

5.2	Action interface	19
5.3	Plugin architecture	19
5.3.1	AuctionBehaviorPluginBase	19
5.3.2	AuctionItemPluginBase	20
5.3.3	Greedy Sequential plugin	20
5.4	Using the CA Gateway	20
5.5	Using the State Interface	21
5.6	Using the Knowledge Base Interface	21
5.7	Python API	21
6	Collision Avoidance Behavior	23
6.1	Overview: consensus and coalition formation as collective models	23
6.2	Action interface	23
6.3	Pairwise path lock plugin	23
6.4	Using the CA Gateway	24
6.5	Using the State Interface	25
6.6	Motion orchestration	25
7	Collective Awareness in the Inspection Scenarios	27
7.1	Normal operation: the complete inspection loop	27
7.2	UC1: Robot failure and swarm replanning	27
7.3	UC2: Emergency landing with collision avoidance	28
7.4	UC3: Discrepancy detection through collective sensing	28
7.5	UC4: Assurance of valid images	28
8	Testbed Integration	29
8.1	End-to-end component interaction	29
8.2	Configuration	29
9	Conclusions	31

■ 1 Introduction

1.1 Context and motivation

The CORESENSE project (Horizon Europe #101070254) aims to endow autonomous robotic systems with a hybrid cognitive architecture that combines reactive and deliberative reasoning grounded in explicit models. Work Package 7 contributes the Drone Inspection Testbed (TB2): a multi-drone system that autonomously surveys a photovoltaic (PV) power plant, allocates inspection targets among itself, and coordinates its motion.

Deliverable D7.1 [Fernandez-Cortizas et al., 2023b] established the operational concept and stakeholder requirements for TB2, identifying four families of use cases: robot failure and swarm replanning (UC1), emergency landing with dynamic path negotiation (UC2), discrepancy detection in collective sensing (UC3), and assurance of valid inspection images (UC4). Deliverable D7.2 [Fernandez-Cortizas et al., 2023c] translated these into system requirements and proposed an initial Aerostack2-based architecture, identifying the three extensions that CoreSense technologies had to provide:

1. **Inter-agent communication** — drones must exchange structured modelets (bids, lock requests, grants) without a centralised broker.
2. **Knowledge grounding** — task assignments and mission events must be recorded in and retrieved from the CORESENSE knowledge base.
3. **Safe multi-robot navigation** — simultaneous access to overlapping flight corridors must be arbitrated before motion begins.

Deliverable D7.3 [Fernandez-Cortizas et al., 2025] elaborated the Collective Awareness Architecture that addresses these needs, defining the Collective Awareness Module and its three subsystems: *Agent Representation* (the agent’s self-model), *Organisation Handling* (collective models and role assignment), and *Communication Interface* (intelligible modelet exchange).

This document, D7.4, reports on the concrete implementation of those three subsystems in Aerostack2.

1.2 Aerostack2 and the cognitive module correspondence

Aerostack2 [Fernandez-Cortizas et al., 2023a] is an open-source, modular framework for autonomous aerial systems built on ROS 2. Its component model separates concerns into *behaviors* (action servers), *state estimators*, *motion controllers*, and *platforms*. Behaviors are the primary locus of robot intelligence: each encapsulates a robot skill as a ROS 2 action server backed by a loadable plugin.

The CORESENSE architecture defines cognitive modules as *subsystems that realise the modelet-engine concept*: they maintain *models* of relevant phenomena and apply *engines* (inference al-

gorithms) to those models to derive outputs. The inputs and outputs communicated between modules are called *modelets*. This maps directly onto Aerostack2 behaviors: the plugin is the engine, the behavior's internal state is the model, and the typed messages it exchanges are the modelets. Chapter 2 makes this correspondence precise.

1.3 Document structure

Chapter 2 establishes the correspondence between Aerostack2 behaviors and CORESENSE cognitive modules. Chapter 3 presents the Collective Awareness middleware—the CA Gateway and CA Client—as the implementation of the D7.3 Communication Interface subsystem; it also introduces the modelet types that flow through it. Chapter 4 explains the Knowledge Base Interface, the layer through which behaviors and mission scripts interact with the CORESENSE knowledge base. Chapters 5 and 6 then describe the Auction Behavior and the Collision Avoidance Behavior, showing in each case how they use the gateway, the knowledge base, and the agent self-model. Chapter 7 traces each D7.1 use case to the components that enable it. Chapter 8 summarises component interaction and configuration.

■ 2 Behaviors as Cognitive Modules

2.1 The modelet-engine concept in CORESENSE

The CORESENSE architecture defines cognitive modules as subsystems that realise the *modelet-engine concept*: they maintain *models*—representations of relevant phenomena, whether numerical, symbolic, or hybrid—and apply *engines*—inference or computational algorithms—to those models to derive relevant information or actions. The typed data items communicated between cognitive modules, and between agents, are called *modelets*.

D7.3 identifies several *collective models* that require inter-agent modelet exchange: social choice, fair division, auctions, negotiation, coalition formation, and consensus. These models take the individual beliefs or values of the agents as input and produce a collective stance (an assignment, an agreement, a shared belief) that no single agent could derive alone. For instance, the *auctions* collective model takes the bids of all agents as input and produces a winner assignment; the *consensus* model takes individual beliefs and produces an agreed value.

Table 2.1 lists the modelets that D7.3 Table 5.1 associates with each collective model, alongside their concrete implementation in this work.

Table 2.1: D7.3 collective models, their modelets, and their implementation

Collective model	D7.3 modelet	Implementation
Auctions	Bids of each agent / winner	<code>Bid / KB is_assigned_to</code>
Coalition formation	Coalition structure / payoff split	<code>CAPathLockRequest / CAPathLockGrant</code>
Consensus	Individual beliefs / agreed value	<code>CAPathLockRequest / access order</code>

2.2 Aerostack2 behaviors as cognitive module realizations

An Aerostack2 behavior is a ROS 2 action server that exposes a well-defined goal, feedback, and result interface and manages execution autonomously from goal acceptance to termination. The behavior framework supports plugin architectures: the behavior server provides scaffolding (action interface, lifecycle, topic wiring) while the plugin provides the *engine*.

The correspondence with CORESENSE cognitive modules is direct:

Models The behavior’s internal state constitutes its model. The Auction Behavior holds cost estimates computed from the drone’s current pose and the geometric description of inspection targets. The Collision Avoidance Behavior holds the drone’s planned path and the peer paths received via lock requests.

Engines The behavior plugin is the engine. The `greedy_sequential` plugin computes costs,

broadcasts bids, and resolves conflicts deterministically. The `pairwise_path_lock_plugin` computes path-to-path distances and arbitrates access using lexicographic priority.

Modellets Each behavior exchanges typed modellets through the CA Gateway. The Auction Behavior sends and receives `Bid` and `StartAuction` messages. The Collision Avoidance Behavior sends and receives `CAPathLockRequest`, `CAPathLockGrant`, and `CAPathLockRelease` messages.

Table 2.2 maps the D7.3 Collective Awareness Module subsystems to the implemented software components.

Table 2.2: D7.3 CA Module subsystems and their implementations

D7.3 subsystem	CORESENSE concept	Implementation
Communication Interface	Intelligible modellet exchange	<code>as2_ca</code> Gateway + Client
Agent Representation	Self-model (pose, battery, ...)	<code>StateInterface</code>
Organisation Handling	Auctions collective model	Auction Behavior (<code>greedy_sequential</code>)
Organisation Handling	Consensus / coalition	Collision Avoidance (<code>pairwise_path_lock</code>)
Knowledge integration	Mission-level fact grounding	<code>KBInterface</code> + <code>kb_monitor</code>

■ 3 Collective Awareness Middleware

3.1 The Communication Interface subsystem

The Communication Interface subsystem of the D7.3 Collective Awareness Module must enable agents to communicate *intelligibly*: it is not enough to transmit bytes; sender and receiver must share the same meaning for the transmitted information. Concretely, this means:

- each agent must have a unique, addressable communication endpoint;
- the system must route an incoming modelet to the cognitive module that registered to handle its type;
- agents must be discoverable without prior configuration of the fleet size.

The `as2_ca` package implements this subsystem through two complementary design principles that govern the entire architecture.

Separation of concerns between use and transport. Any behavior that wishes to participate in a collective process should be able to do so through a simple, uniform interface—declare what modelet types to receive and send modelets to named peers—without knowing anything about how that delivery is performed. The transport responsibility (peer discovery, routing, serialisation) is entirely encapsulated in a dedicated gateway node that runs alongside the existing drone stack.

Non-invasive integration. The gateway extends the system’s collective capability without modifying any existing component. A drone that does not require inter-agent communication is entirely unaffected. Behaviors that do opt in through a thin client library without disrupting existing single-drone functionality. New collective behaviors can therefore be introduced incrementally while all pre-existing Aerostack2 functionality remains unmodified.

These two principles are realised by two complementary artifacts: the **CA Gateway node**, which owns all inter-agent transport, and the **CA Gateway Client**, a lightweight library embedded by behaviors to participate in collective processes. Figure ?? shows how they interact.

Compared to direct use of ROS 2 cross-namespace topics, the CA Gateway operates at a higher level of abstraction. Raw DDS communication requires per-message-type topic definitions, static publisher–subscriber wiring, and explicit reconnection upon fleet reconfiguration. Consulting the ROS 2 graph at runtime to discover active nodes also incurs repeated query overhead that grows with fleet size. The CA Gateway Client abstracts this to two fleet-size-agnostic calls—*register* and *forward*—applicable uniformly across coordination protocols without per-protocol wiring; behavior code is identical in simulation (shared machine) and field deployment (Wi-Fi network). Table ?? summarises the key differences.

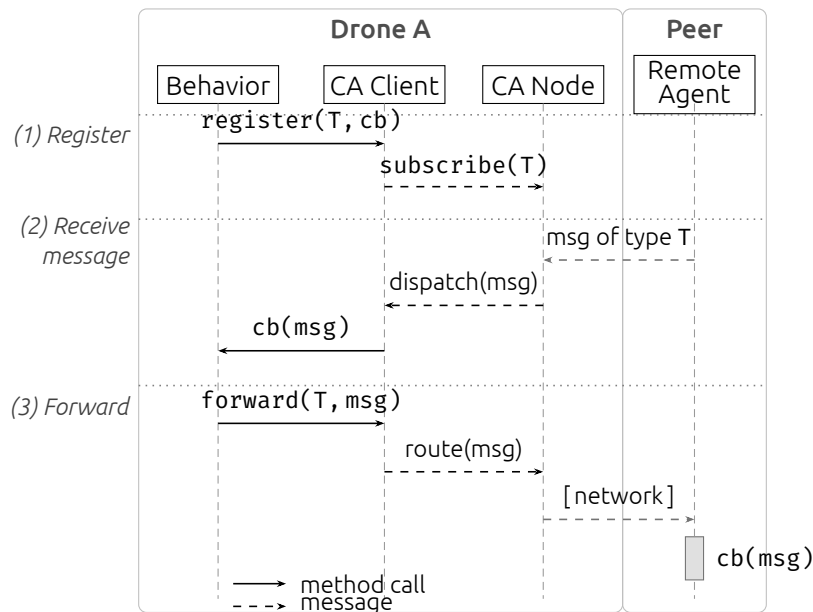


Figure 3.1: CA Gateway message flow. Solid arrows denote method invocations (Behavior–CA Client); dashed arrows denote messages (CA Client–CA Node–Remote Agent). (1) *Register*: a behavior declares callback `cb` for model type `T`; the CA Client subscribes at the node level. (2) *Receive message*: when a peer sends a model type `T`, the CA Node dispatches it to the CA Client, which invokes `cb`. (3) *Forward*: a behavior calls `forward`; the CA Node serialises and routes the model type to all registered peers. In all cases the behavior has no knowledge of peer addresses or network topology.

Table 3.1: CA Gateway vs. direct ROS 2 DDS cross-namespace communication.

Dimension	CA Gateway	Raw ROS 2 DDS
Discovery overhead	Peer registry resolved once at send time	Graph queried per message type at runtime
Naming coupling	Typed registry; no topic names in behavior code	Per-message-type topic names hard-coded
Transport flexibility	Backend swappable (DDS, Zenoh, MQTT)	Tied to DDS
Fleet reconfiguration	Handled automatically by CA Gateway node	Explicit publisher/subscriber rewiring required

3.2 Message types: the modelet vocabulary

Before describing the gateway in detail, it is useful to introduce the message types that flow through it. These types, added to `as2_msgs`, are the concrete ROS 2 representation of the modelets described in D7.3.

3.2.1 Transport envelope

InterAgentMessage The carrier for any typed modelet on the inter-drone channel. It contains a `sender` field (the originating drone namespace), a `type` tag identifying the modelet kind, and a raw `uint8[] data` field carrying the serialised ROS 2 payload.

LocalGenericMessage The intra-agent dual of **InterAgentMessage**, used between a behavior and its drone's gateway. It carries a list of `agents` (target recipients or original senders), a `type` tag, and the same raw `data` field.

The gateway translates between these two representations: outbound **LocalGenericMessage** messages from behaviors are wrapped into **InterAgentMessage** and routed to peers; inbound **InterAgentMessage** packets from peers are unwrapped and delivered as **LocalGenericMessage** to the registered local handler.

3.2.2 Auction modelets

These messages implement the modelets for the *auctions* collective model (D7.3 Table 5.1).

AuctionItem A single item to be auctioned. Carries a `name` string, a `features` float64 array, and a `feature_names` string array, encoding any quantity relevant to cost computation (target coordinates, priority weights, etc.).

AuctionItemArray A container for multiple **AuctionItem** messages, also carrying the name of the item-type plugin to use for cost computation.

StartAuction Sent by the auctioneer to all participants; carries the item list and the set of bidder namespaces.

Bid The core auction modelet—the *bid of each individual agent* of D7.3 Table 5.1. It carries the `name` array of the items the agent has evaluated and an `amounts` float64 array of the corresponding costs.

3.2.3 Path-lock modelets

These messages implement the modelets for the *coalition formation* and *consensus* collective models: the individual path plans and the lock grants that encode the collectively agreed access schedule.

PoseStampedWithID A `geometry_msgs/PoseStamped` augmented with a string `id`, used to identify waypoints in a planned path.

CAPathLockRequest Sent by a drone requesting exclusive access to a path corridor. Contains the `requester_id`, a monotonically increasing `req_id`, and the `path` as a **PoseStampedWithID** array.

CAPathLockGrant Sent by each peer to acknowledge a lock request. Contains the `granter_id`, the `requester_id`, and the matching `req_id`. Receiving a grant from every peer constitutes the consensus that the requesting drone may proceed.

CAPathLockRelease Sent by a drone on finishing a locked corridor, allowing deferred peers to proceed.

3.3 CA Gateway node

The **CA_Gateway** node (package **as2_ca**) is the per-drone communication hub that owns all inter-agent traffic. It continuously scans the active network, creates forwarding channels to newly discovered peers, and prunes channels for peers that have disappeared. This scan provides both runtime peer discovery and implicit failure detection: a crashed or disconnected drone ceases to be reachable within one discovery cycle, and its channel is automatically removed. Concretely, it performs three roles:

1. **Peer discovery.** A one-second timer scans the live ROS 2 topic graph for topics whose name ends in **/gateway_in**. Each newly found topic is added to an internal map of peer publishers; publishers whose topics have no subscribers are pruned. No prior knowledge of the fleet size is needed, and dynamic joins and departures are handled automatically, implementing the Swarm Size Awareness requirement IT-FR-028 from D7.2.

```

10 void CA_Gateway::check_peers()
11 {
12     const std::string suffix = "/" + std::string(INCOMING_TOPIC_SUFFIX);
13     auto topics = this->get_topic_names_and_types();
14     for (auto & [name, types] : topics) {
15         if (name ends with suffix && name != own_incoming_topic) {
16             std::string peer_id = strip_prefix_and_suffix(name);
17             if (not already known)
18                 create publisher to name and remember peer_id;
19         }
20     }
21     prune peers with no subscribers;
22 }

```

2. **Outbound forwarding.** The gateway subscribes to its drone's **gateway_out** topic. When a behavior publishes a **LocalGenericMessage** there, the gateway serialises it into an **InterAgentMessage** and publishes it to the **gateway_in** topic of each named recipient—using the publisher map built by peer discovery.
3. **Inbound demultiplexing.** The gateway subscribes to its own **{namespace}/gateway_in** topic. When an **InterAgentMessage** arrives from a peer, it looks up the registered handler for the message's **type** tag and forwards a **LocalGenericMessage** to the corresponding local topic. This implements the D7.3 requirement to *distribute incoming modellets to the corresponding cognitive modules*.

3.4 CA Gateway Client

Behaviors do not interact with the gateway node directly. Instead, each behavior instantiates a **CAGatewayClient** (header-only, **as2_ca/ca_gateway_client.hpp**), a lightweight library that wraps the gateway's registration service, the **gateway_out** publisher, and type-parametric deserialisation into a clean, type-safe API. It exposes two operations that directly mirror the sequence diagram in Figure ??:

- **register** declares interest in a modellet type and provides a callback invoked on arrival (step 1 in Figure ??).

- **forward** sends a typed modellet to a named set of peers (step 3 in Figure ??).

From the behavior's perspective, peer addresses, network topology, and transport details are completely hidden.

Registering a modellet handler announces to the gateway which message type the behavior handles under a given tag, and provides the callback to invoke on receipt:

```

10 client_.register_module<as2_msgs::msg::Bid>(
    "bid",           // type tag
12  "auction_behavior", // module name
    [this](const as2_msgs::msg::Bid & msg, const std::string & sender) {
14      auction_plugin_->on_bid_received(msg, sender);
    });

```

Sending a modellet serialises a typed ROS 2 message and routes it to a named list of peers:

```

10 client_.forward_IA_msg<as2_msgs::msg::Bid>(bid, "bid", participants_);

```

Internally, `forward_IA_msg` uses `rclcpp::Serialization<T>` to pack the message bytes into a `LocalGenericMessage` and publishes it to `gateway_out`. The CA Gateway node then routes the packet to each named peer's `gateway_in` topic, following the path shown in step 3 of Figure ??.

This two-level design—gateway node for transport, client library for type safety—cleanly separates the communication concern (routing, addressing, fleet management) from the application concern (which modellet type to send, what to do on receipt), mirroring the layered structure of the Communication Interface subsystem in D7.3.

■ 4 Knowledge Base Interface

The CORESENSE knowledge base (`knowledge_core`) stores and reasons over RDF triples and emits events when registered patterns are satisfied. It is the persistent, semantic integration layer that connects individual agent observations to collective reasoning: a fact asserted by one drone’s behavior can trigger a mission-level reaction in another drone’s Python monitor, implementing the information sharing requirement IT-FR-027 from D7.2. Two complementary interfaces connect Aerostack2 to it.

4.1 C++ KBInterface

The `as2::KBInterface` class (header `as2_core/kb_interface.hpp`) provides behaviors with a synchronous-style API for asserting, retracting, and querying facts:

`add_fact(subj, pred, obj)` Publishes a space-separated triple to the `kb/add_fact` topic.

`remove_fact(subj, pred, obj)` Publishes to `kb/remove_fact`.

`query_kb(clauses, variables)` Calls the `kb/query` service and returns the first binding map.

`query_kb_all(clauses, variables)` Returns all binding maps.

`register_event_handler(topic, handler)` Subscribes to a KB event topic and invokes a callback on each trigger.

Service calls are handled on a dedicated background thread backed by a `SingleThreadedExecutor`, so `query_kb` and `query_kb_all` can block safely from inside a ROS 2 callback without re-entering the calling node’s executor. This allows behaviors to treat knowledge-base queries as synchronous operations even though they cross node boundaries.

4.2 Python KB Monitor

Mission-level logic—the layer that decides when to trigger replanning, when to resume execution, and when to ground a failing drone—is typically written in Python. The `as2_python_api.kb_monitor` sub-package provides two complementary tools for this layer.

4.2.1 KBMonitorNode

`KBMonitorNode` is a config-file-driven ROS 2 node. Its JSON configuration lists handler entries, each specifying:

- **`patterns`** — RDF triple patterns with `?`-prefixed variables;

- `one_shot` — whether to remove the handler after the first trigger;
- `func` — the Python module path and function name to invoke on each trigger.

Each handler receives a list of variable-binding dicts and a `KBHandlerContext` exposing `query()`, `add_fact()`, `remove_fact()`, `publish_mission_update()`, the latest `mission_status`, and the latest `pose`.

```

10 {
11     "kb_namespace": "kb",
12     "handlers": [
13         {
14             "id": "assignment_done",
15             "patterns": ["?drone is_assigned_to ?target"],
16             "one_shot": false,
17             "func": {
18                 "path": "/path/to/handlers.py",
19                 "name": "handlers_module",
20                 "func_name": "on_assignment"
21             }
22         }
23     ]
24 }
```

4.2.2 KBMissionMonitor

`KBMissionMonitor` is a programmatic base class for Python mission scripts that need to subscribe to KB events and drive the mission interpreter from the same process:

```

10 rclpy.init()
11 monitor = KBMissionMonitor('drone0')
12
13 def on_target_assigned(bindings):
14     target = bindings[0]['target']
15     msg = MissionUpdate()
16     msg.action = MissionUpdate.RESUME
17     monitor.publish_mission_update(msg)
18
19 monitor.subscribe_to_event(
20     ['?drone is_assigned_to ?target'],
21     on_target_assigned)
22
23 rclpy.spin(monitor)
```

Both tools use a `_QueryHelper` that spins a dedicated `SingleThreadedExecutor` in a daemon thread, making synchronous `query()` calls safe from inside event callbacks.

■ 5 Auction Behavior

5.1 Overview: auctions as a collective model

Task allocation in the inspection scenario is an instance of the *auctions* collective model identified in D7.3: the auctioneer drone publishes a set of inspection targets (items), all participating drones simultaneously compute and broadcast their costs (bids), and each drone resolves conflicts locally until a stable assignment emerges. Because every agent adopts an honest stance—its bid reflects its actual cost to reach each target—this is a fair-division auction.

The `as2_auction_behavior` package implements this collective model as an Aerostack2 behavior server backed by two plugin hierarchies. The behavior uses three of the components described above:

- the **CA Gateway** to distribute the item list and exchange bid modelets;
- the **State Interface** to read the drone’s current pose for cost computation;
- the **KB Interface** to record the assignment as facts in the knowledge base.

This behavior satisfies D7.2 requirements IT-FR-002 (collaborative planning) and IT-FR-003 (system replanning): when a drone failure is detected and its targets become unallocated, the auctioneer triggers a new auction round over only the uncovered targets.

5.2 Action interface

The behavior exposes the `as2_msgs/action/Auction` action. The goal specifies:

- `name` — a unique auction identifier;
- `elements` — the `AuctionItem` modelets to allocate;
- `type` — the auction strategy plugin (e.g. `greedy_sequential`);
- `bidders` — the namespaces of all participating drones.

Feedback reports which items have been assigned so far; the result returns the complete assignment map.

5.3 Plugin architecture

5.3.1 AuctionBehaviorPluginBase

The abstract base class defines the engine interface:

- `on_activate(goal)/on_deactivate()` — lifecycle hooks;
- `on_auction_items_received(msg, agent_id)` — called when the item-list modelet arrives from the auctioneer via the CA Gateway;
- `compute_bid()` — queries the State Interface for the current pose, calls the item plugin for each target, and returns a cost vector;
- `update(bid, agent_id)` — integrates a peer's bid modelet and re-runs conflict resolution;
- `check_convergence()` — returns `true` when all bids have been received;
- `get_global_assignment()` — returns the item→winner map.

5.3.2 AuctionItemPluginBase

A second plugin type abstracts cost computation for a specific item kind. This separation keeps the auction strategy agnostic about item geometry or semantics. The `coordinate_item` plugin computes Euclidean distance from the drone's current pose to a 3-D coordinate target.

5.3.3 Greedy Sequential plugin

The `greedy_sequential` plugin implements the following protocol:

1. On receiving the item list, compute costs for every item and broadcast a `Bid` modelet to all participants via the CA Gateway.
2. On receiving a peer's `Bid` modelet, store it and run conflict resolution: assign each item to the agent with the lowest cost; break ties lexicographically by agent namespace.
3. Declare convergence once a bid has been received from every participant (including self).

Because every agent applies the same deterministic rule to the same set of bids, local and global assignments converge identically without additional communication rounds.

5.4 Using the CA Gateway

The `greedy_sequential` plugin uses `CAGatewayClient` for all modelet exchange. On activation the behavior registers two handlers—one for incoming `StartAuction` modelets (auctioneer distributing the item list) and one for incoming `Bid` modelets (peers broadcasting their costs):

```

10 client_.register_module<as2_msgs::msg::StartAuction>(
    "start_auction", "auction_behavior",
12     [this](const as2_msgs::msg::StartAuction & msg, const std::string &) {
        auction_plugin->on_auction_items_received(msg.items, msg.auctioneer);
14     });

16 client_.register_module<as2_msgs::msg::Bid>(
    "bid", "auction_behavior",
18     [this](const as2_msgs::msg::Bid & msg, const std::string & sender) {
        auction_plugin->on_bid_received(msg, sender);
20     });

```

When `compute_bid()` produces a cost vector, the plugin broadcasts it to all bidders:

```

10 client_.forward_IA_msg<as2_msgs::msg::Bid>(bid, "bid", bidders_);

```

5.5 Using the State Interface

Before computing bids, the behavior configures a `StateInterface` instance on its ROS 2 node to subscribe to the drone's pose topic:

```
10 state_interface_.configure(node_ptr,
    {as2_names::topics::self_localization::pose});
12
13 // Inside compute_bid():
14 auto pose = state_interface_.get_value<geometry_msgs::msg::PoseStamped>(
    as2_names::topics::self_localization::pose);
16 double cost = coordinate_item_plugin_->compute_cost(pose, item);
```

The State Interface is the Agent Representation subsystem of D7.3: it provides the self-model data (the agent's own position) that the engine (greedy-sequential) uses to compute the model output (a cost vector). It is pre-configured with three entries—`self_localization/pose` (`PoseStamped`), `self_localization/twist` (`TwistStamped`), and `sensor_measurements/battery` (`BatteryState`)—and new entries can be added via a macro without modifying the class.

5.6 Using the Knowledge Base Interface

After convergence, `AuctionBehavior::publish_results_to_kb()` grounds the assignment in the CORESENSE knowledge base:

```
10 for (const auto & [item, winner] : assignment) {
    kb_interface_.add_fact(winner, "is_assigned_to", item);
12 }
    kb_interface_.add_fact(own_namespace_, "auction_id", auction_id_);
14 kb_interface_.add_fact(auction_id_, "auction_status", "done");
```

A `KBMonitorNode` configured with the pattern `?drone is_assigned_to ?target` detects these facts as they appear and publishes a `MissionUpdate` to resume each drone's mission interpreter. This closes the loop: the collective model (auction) produces modelets (bids) exchanged through the CA Gateway, the engine resolves them into an assignment, and the KB Interface makes that assignment visible to the reactive mission layer.

5.7 Python API

The `AuctionModule` class extends both `ModuleBase` and `AuctionBehavior` so that mission scripts can launch an auction with a single call:

```
10 drone.auction(
    name='inspection_round_1',
12    elements=targets,
    auction_type='greedy_sequential::Plugin',
14    bidders=['drone0', 'drone1', 'drone2'])
```


■ 6 Collision Avoidance Behavior

6.1 Overview: consensus and coalition formation as collective models

After the auction assigns inspection targets, each drone plans a path and begins navigating. If two paths share a corridor closer than a safety threshold, simultaneous execution risks a mid-air collision. The collision avoidance behavior addresses this with a *coalition formation* and *consensus* collective model: drones that intend to traverse overlapping corridors temporarily form a coalition and collectively agree—by exchanging lock requests and grants—on an access order.

In D7.3 terms, the *consensus* being reached is: “*drone i will traverse this corridor before drone j* ”. The individual beliefs communicated as modelets are the `CAPathLockRequest` messages; the grants are the vote of each peer that the requester may proceed; the consensus is established when all peers have granted.

The behavior uses two of the shared components:

- the **CA Gateway** to broadcast lock requests and receive grants and releases;
- the **State Interface** to prepend the drone’s current pose to the planned path before locking.

It satisfies D7.2 requirements IT-FR-031 (trajectory collective-awareness) and IT-FR-032 (security distance), and is the primary mechanism enabling the UC2 emergency landing scenarios from D7.1.

6.2 Action interface

The `as2_msgs/action/CollisionAvoidance` goal carries:

- `path` — the planned waypoints as `PoseStampedWithID[]`;
- `safety_distance` — the minimum acceptable separation in meters.

The behavior acquires the path lock, executes the path by delegating to the existing `GoToWaypoint` action, and releases the lock on completion or cancellation.

6.3 Pairwise path lock plugin

The `pairwise_path_lock_plugin` implements the following state machine for each drone i :

IDLE No active goal. All peer requests are granted immediately.

REQUESTING Drone i has broadcast a `CAPathLockRequest` and is waiting for grants from all known peers. Requests from conflicting peers with lower lexicographic priority are deferred.

HOLDING All grants received. Motion may begin.

RELEASING Execution finished. A `CAPathLockRelease` is broadcast and deferred grants are flushed.

Conflict detection uses the minimum distance between two piecewise-linear paths (polylines). `path_geometry::min_polyline_distance()` iterates over all pairs of segments; if the result is below `safety_distance`, the paths conflict.

Tie-breaking uses lexicographic order of drone namespaces: the drone with the smaller name has priority. When two conflicting drones request simultaneously, the one with the larger name grants immediately while the one with the smaller name waits.

Resilience: if a peer releases before its grant arrives, its silence is treated as an implicit grant; if a peer releases before having been granted, the deferred entry is discarded.

6.4 Using the CA Gateway

On activation the behavior registers handlers for each path-lock modelet type via the CA Gateway Client:

```

10 client_.register_module<as2_msgs::msg::CAPathLockRequest>(
    "path_lock_request", "collision_avoidance",
12     [this](const as2_msgs::msg::CAPathLockRequest & req,
            const std::string &) { on_request(req); });
14
16 client_.register_module<as2_msgs::msg::CAPathLockGrant>(
    "path_lock_grant", "collision_avoidance",
18     [this](const as2_msgs::msg::CAPathLockGrant & grant,
            const std::string &) { on_grant(grant); });
20
22 client_.register_module<as2_msgs::msg::CAPathLockRelease>(
    "path_lock_release", "collision_avoidance",
    [this](const as2_msgs::msg::CAPathLockRelease & rel,
            const std::string &) { on_release(rel); });

```

When a drone wants to enter the REQUESTING state, it broadcasts a `CAPathLockRequest` to all known peers via the CA Gateway:

```

10 as2_msgs::msg::CAPathLockRequest req;
    req.requester_id = own_id_;
12 req.req_id        = ++req_id_counter_;
    req.path         = locked_path_;
14 client_.forward_IA_msg<as2_msgs::msg::CAPathLockRequest>(
    req, "path_lock_request", known_peers_);

```

The full grant/defer logic is:

```

10 bool should_grant = true;
    if (state_ == HOLDING && conflicts(req.path, locked_path_))
12     should_grant = false;
    else if (state_ == REQUESTING && conflicts(req.path, locked_path_))
14     should_grant = req.requester_id < own_id_; // lex priority

```

```

16 if (should_grant) {
    send_grant(req.requester_id, req.req_id);
18     peer_held_paths_[req.requester_id] = req.path;
} else {
20     deferred_.push_back({req.requester_id, req.req_id, req.path});
}

```

6.5 Using the State Interface

On goal activation, before broadcasting the lock request, the behavior reads the drone's current pose from the State Interface and prepends it to the goal path:

```

10 state_interface_.configure(node_ptr,
    {as2_names::topics::self_localization::pose});
12
13 auto pose = state_interface_.get_value<geometry_msgs::msg::PoseStamped>(
14     as2_names::topics::self_localization::pose);
16 locked_path_.insert(locked_path_.begin(),
    pose_to_stamped_with_id(pose, "current"));

```

This ensures that the lock covers the full corridor from the drone's current position to the first planned waypoint—not just from the first planned waypoint onward—which is the only correct interpretation of “reserving the path” at the moment of invocation.

6.6 Motion orchestration

The behavior orchestrates the full acquire-execute-release lifecycle by delegating motion to the existing `GoToWaypoint` action client:

1. **REQUESTING_LOCK** — broadcast `CAPathLockRequest` to all known peers; wait until a `CAPathLockGrant` has been received from every peer.
2. **HOLDING_LOCK** — send the `GoToWaypoint` goal; transition to `EXECUTING_MOTION`.
3. **EXECUTING_MOTION** — wait for the `GoToWaypoint` result.
4. **RELEASING_LOCK** — broadcast `CAPathLockRelease`; flush deferred grants; return success.

On `on_pause()`, the in-flight `GoToWaypoint` is cancelled and the lock is released so peers are not blocked while the drone is paused. On `on_resume()`, the lock is re-acquired from scratch before motion restarts.

■ 7 Collective Awareness in the Inspection Scenarios

This chapter traces each use case family defined in D7.1 and D7.2 to the components that enable it.

7.1 Normal operation: the complete inspection loop

The default scenario (D7.1 Section 7.1) requires the swarm to divide the inspection area, navigate safely to each panel, take images, and return. The implemented components cover steps 5 and 6 of that scenario (task distribution and safe trajectory planning):

1. The mission script calls `drone.auction( ... )` with the inspection targets.
2. Each drone's `AuctionBehavior` reads its current pose from the State Interface, computes costs via the `coordinate_item` plugin, and broadcasts a `Bid` modelet through the CA Gateway.
3. Every drone runs `greedy_sequential` on the received bids; the auction converges and results are asserted in the knowledge base.
4. The KB monitor detects the `is_assigned_to` triples and resumes each drone's mission interpreter.
5. Each drone invokes `CollisionAvoidanceBehavior` with its planned path. If paths conflict, the pairwise lock protocol reaches consensus on access order before motion begins.
6. Drones navigate to their targets; on completion, path locks are released and deferred peers proceed.

This loop satisfies IT-FR-002, IT-FR-027, IT-FR-031, and IT-FR-032 from D7.2.

7.2 UC1: Robot failure and swarm replanning

UC1 (D7.1 Section 7.3) covers battery decay (UC1.1) and GPS loss (UC1.2) scenarios:

- **Failure detection:** The failing drone's battery level is cached by its State Interface and read by the Power Management cognitive function. When an abnormal decay is detected, the fact `drone0 battery_status critical` is asserted in the knowledge base.
- **Mission reaction:** A KB monitor handler triggers an emergency landing sequence and publishes a `MissionUpdate` removing the drone from the active fleet.

- **Replanning:** The mission monitor triggers a new auction over the uncovered targets. The `AuctionBehavior` reallocates them among the remaining drones, computing fresh costs from each drone's current pose via the State Interface.

This satisfies IT-FR-003 (system replanning) and IT-FR-016 (failure resilience).

7.3 UC2: Emergency landing with collision avoidance

UC2 (D7.1 Section 7.4) addresses a drone landing inside the inspection area. D7.1 states explicitly that *"drones must maintain a safe distance at all times, so making an emergency landing inside the inspection area means changing the trajectory of nearby drones"* (UC2.1, step 3).

The collision avoidance behavior handles this directly: the failing drone invokes `CollisionAvoidanceBehavior` with its emergency landing path and broadcasts a `CAPathLockRequest`. Peers in HOLDING state grant the request immediately (they release and yield); peers in REQUESTING state grant if the failing drone has lexicographic priority. Once the failing drone broadcasts `CAPathLockRelease`, deferred peers resume.

This is the scenario described in D7.3 Section 5.3 (*Social Choice for Emergency Landing*), where the Organisation Handling subsystem coordinates agent behaviour around an emergency using modelets. The `pairwise_path_lock_plugin` is the concrete Aerostack2 implementation of that coordination pattern.

7.4 UC3: Discrepancy detection through collective sensing

UC3 (D7.1 Section 7.5) shows the swarm detecting faulty drone behaviour by identifying discrepancies in shared observations—a panel position estimate that deviates from the plant model, or a wind estimate that differs from peers.

Each drone asserts its observations as KB triples. When the map generation module detects a discrepancy, it asserts `drone0 localization_status suspect`. A KB monitor handler on that pattern queries all recent observations from that drone; if multiple inconsistencies accumulate, it asserts `drone0 mission_status grounded` and triggers a replanning auction for the drone's targets.

The CA Gateway is the channel through which peer observations are shared (via `InterAgentMessage`); the KB is the layer where they are integrated into a collective model of the environment. UC3 is the use case that most directly exercises collective awareness in D7.3's sense: the swarm forms a collective environmental model by aggregating individual observations and uses discrepancies in that model to infer agent failures.

7.5 UC4: Assurance of valid images

UC4 (D7.1 Section 7.6) covers individual image quality problems (blur, brightness, artifacts). These scenarios are primarily individual—each drone reasons about its own images—and are handled by the image quality cognitive function external to the components described here.

The KB interface provides the bridge to collective replanning when needed: when the image quality module determines that a panel was not validly inspected, it asserts `panel_X inspection_status invalid`. A KB monitor handler can trigger a partial re-auction covering only the invalid panels, connecting an individual cognitive function to a collective response.

■ 8 Testbed Integration

8.1 End-to-end component interaction

The components described in Chapters 3–6 interact as an integrated collective awareness system. The following traces the complete inspection cycle for a two-drone fleet, cross-referencing each component:

1. A mission script calls `drone0.auction( ... )` with the inspection targets.
2. `drone0`'s `AuctionBehavior` sends a `StartAuction` modelet to `drone1` via the **CA Gateway**.
3. Both drones read their current pose from their **State Interface** and compute costs via the `coordinate_item` plugin.
4. Both drones broadcast `Bid` modelets via the **CA Gateway** and run `greedy_sequential`. The auction converges; results are asserted in the **KB** via `KBInterface`.
5. The **KB monitor** detects the `is_assigned_to` triples and resumes each drone's mission interpreter.
6. Each drone calls `CollisionAvoidanceBehavior`, which reads the current pose from the **State Interface** and prepends it to the goal path.
7. Each drone broadcasts a `CAPathLockRequest` via the **CA Gateway**. If the paths conflict, the drone with the smaller namespace acquires the lock first and begins navigating; the other waits.
8. On reaching its target, each drone broadcasts `CAPathLockRelease` via the **CA Gateway**, allowing the deferred peer to proceed.

8.2 Configuration

Each component is configured via standard ROS 2 YAML parameter files. The key parameters are:

Table 8.1: Key configuration parameters

Component	Parameter	Default
CA Gateway	<code>out_messages_topic</code>	<code>gateway_out</code>
CA Gateway	<code>register_module_service_name</code>	<code>register_module</code>
Auction	<code>plugin_name</code>	<code>greedy_sequential::Plugin</code>
Auction	<code>state_component</code>	<code>[]</code>
Collision Avoidance	<code>plugin_name</code>	<code>pairwise_path_lock_plugin::Plugin</code>
Collision Avoidance	<code>safety_distance</code>	<code>1.5 m</code>

■ 9 Conclusions

This deliverable has reported the implementation of the CORESENSE Drone Inspection Testbed (TB2) in Aerostack2, following the infrastructure-first narrative of the D7.3 Collective Awareness Architecture.

The CA Gateway and CA Client ([as2_ca](#)) implement the D7.3 Communication Interface subsystem: per-agent namespaced topics, automatic peer discovery, and type-dispatched modellet routing provide the intelligible communication channel that collective models require. On top of this, the Knowledge Base Interface ([as2_core](#), [as2_python_api](#)) provides the persistent, semantic integration layer: behaviors assert facts about mission events and collective decisions, and KB monitor handlers react to those facts to drive mission-level replanning.

The Auction Behavior and the Collision Avoidance Behavior then show how Aerostack2 behaviors function as CORESENSE cognitive modules: each maintains models (cost estimates, planned paths, or peer-observed paths), applies an engine (the greedy-sequential algorithm, the pairwise path-lock protocol), and exchanges modellets through the CA Gateway. The Auction Behavior uses the gateway for bid exchange, the State Interface for self-model data (current pose for cost computation), and the KB Interface to ground the assignment in the knowledge base. The Collision Avoidance Behavior uses the gateway for lock-request exchange and the State Interface to prepend the current position to the path before locking.

Together, these components realise the four use-case families of D7.1: swarm replanning after robot failure (UC1) via re-auction triggered by KB events, safe emergency landing (UC2) via the consensus-based path-lock protocol, discrepancy detection through collective sensing (UC3) via CA Gateway exchange and KB integration, and image quality assurance (UC4) via KB-triggered partial re-auction. They satisfy the D7.2 requirements for collaborative planning, system replanning, failure resilience, information sharing, swarm size awareness, trajectory collective-awareness, and security distance.

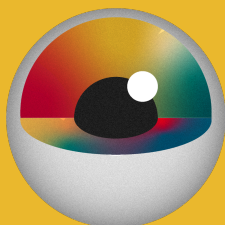
The modular plugin architecture ensures that alternative auction strategies, cost functions, and path-lock protocols can be substituted without modifying the behavior infrastructure, allowing the testbed to evolve as the CORESENSE cognitive module library grows.

Bibliography

- [Fernandez-Cortizas et al., 2025] Fernandez-Cortizas, M., G.P.-Lenza, G., Molina, M., and Sanz, R. (2025). D7.3 – collective awareness architecture. Technical Report CS-095 v1.0, CORESENSE Consortium, Horizon Europe. Contract #101070254.
- [Fernandez-Cortizas et al., 2023a] Fernandez-Cortizas, M., Molina, M., Arias-Perez, P., Perez-Segui, R., Perez-Saura, D., and Campoy, P. (2023a). Aerostack2: A software framework for developing multi-robot aerial systems. In *Proceedings of the 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 9618–9625.
- [Fernandez-Cortizas et al., 2023b] Fernandez-Cortizas, M., Perez-Saura, D., and Molina, M. (2023b). D7.1 – inspection testbed concept and requirement analysis. Technical Report CS-023 v1.0, CORESENSE Consortium, Horizon Europe. Contract #101070254.
- [Fernandez-Cortizas et al., 2023c] Fernandez-Cortizas, M., Perez-Saura, D., and Molina, M. (2023c). D7.2 – inspection testbed specification. Technical Report CS-056 v1.0, CORESENSE Consortium, Horizon Europe. Contract #101070254.



HORIZON
EUROPE



CORESENSE

Title **D7.4 Inspection Testbed Implementation**
Subtitle Collective awareness, task allocation, and collision avoidance in Aerostack2
Author Guillermo GP-Lenza (UPM)
Date 2026/04/28
Reference CS-D7.4
Version 0.1
URL <http://www.coresense.eu/doc/CS-D7.4.pdf>

© 2022-2026 The CORESENSE Horizon Europe Project
coresense.eu